

```

100 | host | {all} | {all} | all | scram-
sha-256
(7 rows)

```

As you can see, the `pg_hba_file_rules` reports all the same information you can find in `pg_hba.conf`, with the line number indicator that tells you from which line a specific rule has been loaded.

Including other files in `pg_hba.conf`

It is possible to include other HBA configuration files into the main `pg_hba.conf` file. PostgreSQL provides three main directives:

- `include_file` includes a specific file in `pg_hba.conf`
- `include_if_exist` includes a specific file but only if it exist; if it does not exist (or was removed), no error will occur
- `include_dir` includes all files specified in the given directory

Thanks to this directive, it is possible to define a set of small configuration files that will be included literally in the HBA configuration as if the administrator had edited the `pg_hba.conf` file directly.

In order to understand where a specific rule comes from, the `pg_hba_file_rules` catalog includes a `file_name` column that reports from which file (and at which line, thanks to `line_number`) a rule has been parsed.

Summary

PostgreSQL allows you to define single users and groups of users, both represented by the SQL concept of roles. When a database connection attempt is made, PostgreSQL processes the connection information through the host-based access control so that it can immediately establish or reject the connection, depending on firewall-like rules. If the connection can be established, the credentials for the role are checked, and at last, the user is granted access.

Users and groups can be fine-tuned in terms of their granted permissions and connection limitations so that you can decide how many resources a single role can consume.

In this chapter, you have seen how to create and manage roles, as well as how to allow single roles to connect to the cluster and to specific databases. In *Chapter 10, Users, Roles, and Database Security*, you will see how to deal with the security properties of users and groups, but before you proceed further, you need to know how PostgreSQL objects can be created and managed.

In the following chapter, you will learn how to interact with the PostgreSQL database using SQL statements.

Verify your knowledge

- What is the aim of the `pg_hba.conf` file?

The `pg_hba.conf` file configures **Host-Based-Access (HBA)**, a set of rules that define how a specific role (either a user or a group) can establish a connection to a specific database from a specific host or source, via a defined protocol. See the *Managing incoming connections at the role level* section for more details.

- How can you inspect the currently loaded HBA rules?

The special catalog `pg_hba_file_rules` provides details about loaded rules. See the *Inspecting `pg_hba.conf` rules* section for more details.

- Does the order of rules within `pg_hba.conf` matter?

Yes, the rules are evaluated from top to bottom, and the first matching rule causes the end of the evaluation. See the *Order of rules in `pg_hba.conf`* section for more details.

- Where can you find information about roles?

The special catalogs `pg_roles` and `pg_authid` provide information about roles. See the *Inspecting existing roles* section for more details.

- How can you add a role to a group or remove it from a group (i.e., another role)?

The `GRANT` statement can add a role to another one, while the `REVOKE` statement can remove the association. See the *Using a role as a group* section for more details.

References

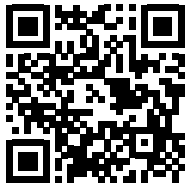
- `CREATE ROLE` statement official documentation: <https://www.postgresql.org/docs/current/sql-createrole.html>
- `DROP ROLE` statement official documentation: <https://www.postgresql.org/docs/current/sql-droprole.html>
- PostgreSQL `pg_roles` catalog details: <https://www.postgresql.org/docs/current/view-pg-roles.html>

- PostgreSQL `pg_authid` catalog details: <https://www.postgresql.org/docs/current/catalog-pg-authid.html>
- PostgreSQL host-based access rule details: <https://www.postgresql.org/docs/current/auth-pg-hba-conf.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



4

Basic Statements

In this chapter, we will discuss basic SQL commands for PostgreSQL; these are **Data Definition Language (DDL)** commands and **Data Manipulation Language (DML)** commands. In basic terms, DDL commands are used to manage databases and tables, and DML commands are used to insert, delete, update, and select data inside databases. In this chapter, we will also dive into the `psql` environment. As you learned in *Chapter 2, Getting to know your cluster*, `psql` can be described as PostgreSQL's shell environment; it is the gate we have to go through in order to start writing commands natively in PostgreSQL. We have to remember that `psql` is always present in any PostgreSQL installation we work with, and it is worth learning since it is such a powerful environment in which to manage our data and our databases.

Basic statements and `psql` are therefore the foundations on which we will build our knowledge of PostgreSQL. Therefore, reading and understanding this chapter is essential to understanding some of the more complex topics we cover later.

Let's start with a list of what we're going to learn in this chapter:

- Setting up our development environment
- Creating and managing databases
- Managing tables
- Understanding basic table manipulation statements

Technical requirements

At this point in the book, we have learned how to install PostgreSQL and how to configure users, but if you haven't read the previous chapters, you can easily start following the next steps using a Docker image as described below.

Using the Docker image

If you want to follow the next steps without installing and configuring PostgreSQL, you can do so easily using the Docker image in the GitHub repository (details on how to set it up are covered in *Chapter 1, Introduction to PostgreSQL*). So, let's start the standalone container as described in *Chapter 1, Introduction to PostgreSQL*, and then execute the following:

```
$ sudo docker exec -it standalone_learn_postgresql_1 /bin/bash
```

After executing this instruction, we will be inside the `standalone_learn_postgresql_1` container in a root shell:

```
root@learn_postgresql:/#
```

Connecting the database

Even if we didn't use a Docker container but used a native PostgreSQL installation as described in *Chapter 1, Introduction to PostgreSQL*, we would reach the same result as above, using the same statement executed as a postgres user:

```
root@learn_postgresql:/# su - postgres
postgres@learn_postgresql:~$ psql
postgres=#
```

Now let's switch on the expanded mode using the `\x` command:

```
postgres=# \x
Expanded display is on.
```

Then let's list all the databases that are present in the cluster:

```
postgres=# \l
List of databases
-[ RECORD 1 ]-----+-----
Name          | forumdb
```

```
Owner          | forum
Encoding       | UTF8
Collate        | en_US.utf8
Ctype          | en_US.utf8
ICU Locale     |
Locale Provider | libc
Access privileges |
```

For space reasons, we have reported only the `forumdb` database imported from the Docker script, but there are also the `template0`, `template1`, and `postgres` databases as we saw in *Chapter 2, Getting to know your cluster*. Finally, let's connect to the `forumdb` database:

```
postgres=# \c forumdb
You are now connected to database "forumdb" as user "postgres".
```

Now that we have finished setting up our development environment, we can move on to creating databases in it.

Creating and managing databases

In this section, we will start by creating our first database, then we will learn how to delete a database and, finally, how to create a new database from an existing one. We will also analyze the point of view of the DBA. We will see what happens behind the scenes when we create a new database and learn some basic functions useful to the DBA to get an idea of the real size of the databases.

Let's see how to create a database from scratch and what happens behind the scenes when a database is created.

Creating a database

To create a database named `databasename` from scratch, you will need to execute this simple statement:

```
CREATE DATABASE databasename;
```



SQL is a case-insensitive language, so we can write all the commands with uppercase or lowercase letters.

Now, let's see what happens behind the scenes when we create a new database. PostgreSQL performs the following steps:

1. Makes a physical copy of the template database, `template1`
2. Assigns the database name to the database just copied

The `template1` database is a database that is created by the `initdb` process during the initialization of the PostgreSQL cluster.

Managing databases

We've just seen how to create databases. In this section, we will see how to manage databases, how to list all the databases present on a cluster, how to create a database starting from an existing database, how to drop a database, and what happens internally, behind the scenes, when we create and drop a database.

Introducing schemas

As reported in *Chapter 1, Introduction to PostgreSQL*: “a database can be organized into namespaces, called schemas. A schema is a mnemonic name that the user can assign to organize database objects, such as tables, into a more structured collection. Schemas cannot be nested, so they represent a flat namespace.” Referring again to *Chapter 1, Introduction to PostgreSQL*, we've learned that there are two kinds of users, **normal users** and **superusers**:

- **Superusers** can do everything across databases and schemas.
- **Normal users** can do operations depending on their privilege set.

PostgreSQL and the public schema

Starting from PostgreSQL 15, PostgreSQL has changed the way to manage the public schema. In this section, we will see how it works. Before PostgreSQL 15, any user was able to perform any DDL operation on the public schema. PostgreSQL 15 introduces the concept of removing global privileges from the public schema.

Starting from PostgreSQL 15:

- A **normal user** will not be able to execute DDL on the public schema.
- A **normal user** will not be able to perform DML on the public schema unless they receive permission from a superuser.

Let's use an example to better explain how this new feature works. We will work as if we were on a PostgreSQL version $\leq 14.x$.

The following are the steps that we will execute (some instructions will be explained later in this book):

1. We will create a normal user called `myuser`.
2. We will connect to the database as the user `myuser`.
3. As `myuser`, we will try to create a new table called `mytable`.

Below, you will find the execution of what is written above:

```
forumdb=# create user myuser with password 'SuperSecret' login;
CREATE ROLE
forumdb=# set role to myuser;
SET
forumdb=> create table mytable(id integer);
ERROR:  permission denied for schema public
LINE 1: create table mytable(id integer);
```

As we can see, a **normal** user cannot create a table (DDL) on a **public** schema.

The `search_path` variable

PostgreSQL has many system variables. One of them is called `search_path`. The `search_path` variable contains the sequence of schemas that PostgreSQL uses to find tables; the `search_path` default value is `$user, public`. This means that first it will search all the tables in the schema that have that name in the user table and then it will search the public schema.

For example, if we have a user called `forum`, and we want to show all the records that are present in a table called `cities`, first PostgreSQL will search the `cities` table in the `forum` schema, and if the `cities` table cannot be found in the `forum` schema, PostgreSQL will search for the `cities` table in the public schema.

The correct way to start working

Let's start from scratch and execute the following steps:

1. As a superuser, let's create a new database called `myforumdb` and connect to it.
2. As a superuser, let's create a new user called `myforum`.
3. As a superuser, let's create a new schema called `myforum` with authorization for the `myforum` user.

4. Let's connect to the database as the myforum user:

```
postgres=# create database myforumdb;
CREATE DATABASE
postgres=# \c myforumdb
You are now connected to database "myforumdb" as user "postgres".
myforumdb=# create user myforum with password 'SuperSecret' login;
CREATE ROLE
myforumdb=# create schema myforum authorization myforum;
CREATE SCHEMA
```

Now let's try to connect to the myforumdb database as the myforum user:

```
postgres@learn_postgresql:/$ psql -U myforum myforumdb
myforumdb=>
```

Let's try to create a new table called mytable as we have done before:

```
myforumdb=> create table mytable(id integer);
CREATE TABLE
```

Now it works! It works because the mytable table has been created inside the myforum schema as we have explained above.



The forumdb database provided with the container is already set up to be used using the forum user, which refers to the forum schema.

Listing all tables

Let's now connect to the forumdb database as the forum user:

```
postgres@learn_postgresql:/$ psql -U forum forumdb
forumdb=>
```

To list all the tables present in the forumdb database, we have to use the psql \dt command. The \dt command makes a list of all the tables present in the forumdb database:

```
forumdb=> \dt
List of relations
```

```

Schema |      Name      | Type | Owner
-----+-----+-----+-----
forum  | categories     | table | forum
forum  | j_posts_tags   | table | forum
forum  | posts          | table | forum
forum  | tags           | table | forum
forum  | users          | table | forum
(5 rows)

```

Making a new database from a modified template

Now that we've learned how to list all tables in a database, let's ensure that any changes made to the `template1` database will be seen by all the databases that will be created later. We will perform these steps:

1. Connect to the `template1` database.
2. Create a table called `dummytable` inside the `template1` database.
3. Create a new database called `dummydb`.

Let's start making the database using the following steps:

1. Connect to the `template1` database:

```

postgres@learn_postgresql:/$ psql template1
template1=#

```

2. As **superuser**, create a table called `dummytable`. For now, we don't need to worry about the exact syntax for creating tables; this will be explained in more detail later on:

```

template1=# create table dummytable (dummyfield integer not null
primary key);
CREATE TABLE

```

3. Use the `\dt` command to show a list of tables that are present in the `template1` database:

```

template1=# \dt
              List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | dummytable     | table | postgres
(1 row)

```